

FEDERAL-SLM-SYSTEM

Federated Document Intelligence Platform

1. Executive Summary

The Federal-SLM-System is an advanced, privacy-preserving document intelligence platform that combines modern AI inference, optical character recognition (OCR), structured data extraction via a Small Language Model (SLM) pipeline, federated machine learning, and blockchain-based audit logging. The system is designed to enable multiple independent organizations to collaboratively train AI models on sensitive documents — without ever sharing raw data.

The architecture consists of six tightly integrated subsystems:

- A React-based frontend dashboard supporting three distinct user roles: Admin, Org (Organization), and Researcher.
- A FastAPI backend providing RESTful endpoints for document ingestion, authentication, training orchestration, schema management, and blockchain query.
- A dedicated PaddleOCR microservice that transforms uploaded image/PDF documents into structured text.
- An SLM (Structured LLM) pipeline backed by Ollama and LangChain that converts raw OCR output into machine-readable JSON records.
- A Federated Learning subsystem using the Flower (flwr) framework with a FedAvg aggregation strategy, enabling privacy-preserving collaborative model training.
- A custom blockchain logger that provides cryptographically signed, immutable audit trails for all key system actions.

Key Innovation

The system's core contribution is enabling multi-organization AI model training on sensitive documents (e.g., medical records)

without centralizing raw data. Each organization retains data sovereignty while contributing to a shared global model.

All actions are logged immutably on a local proof-of-work blockchain, ensuring full auditability.

2. Introduction & Problem Statement

2.1 Background

In domains like healthcare, finance, and legal services, organizations frequently possess large collections of sensitive documents that contain valuable structured information. Extracting this information and using it to train predictive models offers significant utility. However, sharing raw documents across organizational boundaries raises serious privacy, regulatory, and security concerns — particularly under frameworks like HIPAA, GDPR, and ISO 27001.

Traditional centralized machine learning approaches require aggregating all training data in one location, which is often legally impermissible and technically risky. Federated Learning (FL) addresses this by bringing computation to the data rather than data to the computation — training models locally and sharing only model weight updates.

2.2 Problem Statement

The Federal-SLM-System addresses the following challenges simultaneously:

1. How to extract structured, machine-learning-ready records from unstructured document images automatically.
2. How to enable multiple organizations to train a shared model without centralizing sensitive data.
3. How to ensure transparent, tamper-proof audit trails for all document processing and model training events.
4. How to provide role-differentiated access to capabilities (uploading, training, downloading models, viewing audits).

2.3 Scope

This report covers the complete technical architecture of the system as implemented in the repository at <https://github.com/thanushshetty9353/Federal-SLM-System>. It documents the data flows, component designs, API surfaces, database schema, security model, federated learning protocol, and deployment configuration. The primary use case demonstrated in the codebase is cancer diagnostic record extraction from uploaded medical documents.

3. System Architecture

The Federal-SLM-System follows a layered, microservice-oriented architecture. The system can be logically decomposed into four horizontal layers: Presentation, Processing, Intelligence, and Persistence.

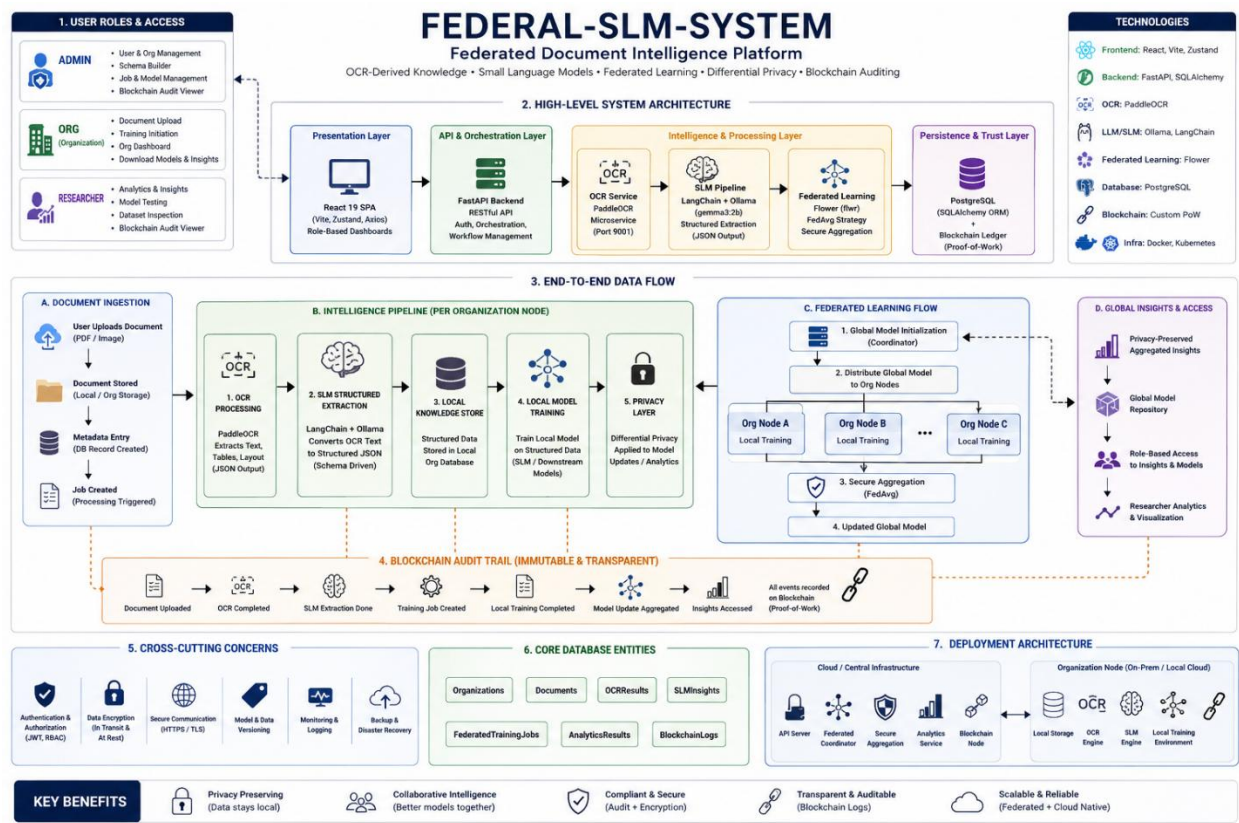


Figure 1: High-Level System Architecture — Six primary subsystems across four logical layers

3.1 Presentation Layer

The frontend is a React 19 Single Page Application (SPA) built with Vite as the build tool. It serves three distinct role-based user experiences:

Role	Key Capabilities
------	------------------

ADMIN	User management, schema builder, job management, model access control, blockchain viewer
ORG (Organization)	Document upload, training initiation, org dashboard, downloads
RESEARCHER	Analytics, model testing, dataset inspection

State management is handled through Zustand, API communication through Axios, and rich interactions are supported by libraries including framer-motion (animations), Recharts (analytics), react-dropzone (file upload), and react-hot-toast (notifications).

3.2 Processing Layer (Backend)

The FastAPI backend serves as the system's central orchestration hub. It exposes a comprehensive REST API and coordinates all operations: authentication, document handling, OCR invocation, SLM extraction, training orchestration, and blockchain logging. It communicates with the database via SQLAlchemy ORM and with the OCR microservice via HTTP.

3.3 Intelligence Layer

This layer encompasses two orthogonal AI systems:

- OCR Service: A standalone FastAPI microservice running PaddleOCR on port 9001, responsible for converting document images to text.
- SLM Pipeline: A LangChain + Ollama integration that performs structured extraction from OCR text, using the gemma3:2b model by default.
- Federated Learning: A Flower (flwr) framework cluster where a central server coordinates model weight aggregation across multiple organization-level clients using FedAvg.

3.4 Persistence Layer

Data is persisted across two storage tiers:

- SQL Database (SQLite for development, PostgreSQL for production): Stores user accounts, document metadata, OCR results, SLM insights, schema configurations, and blockchain logs.
- Filesystem Storage: Stores raw uploaded documents, per-organization JSON/CSV datasets, locally trained model weights (.pth), and the global federated model.

4. Technology Stack

4.1 Frontend Technologies

Library / Framework	Purpose & Notes
React 19.x	Core UI rendering engine — functional components and Hooks
Vite	Development server and production build tooling — fast HMR
React Router	Client-side SPA routing with role-based guards
TailwindCSS	Utility-first CSS — responsive and consistent styling
Zustand	Lightweight global state management (replaces Redux/Context for complex state)
Axios	HTTP client for API communication; supports interceptors for JWT injection
Framer Motion	Declarative animation library for page transitions and UI feedback
Recharts	Composable charting components for analytics dashboards
react-dropzone	Drag-and-drop file upload with MIME validation
jwt-decode	Client-side JWT parsing to extract role/org_id claims
lucide-react	Consistent SVG icon library
react-hot-toast	Non-intrusive toast notification system

4.2 Backend Technologies

Library / Framework	Purpose & Notes
---------------------	-----------------

FastAPI	High-performance async REST framework with automatic OpenAPI docs
Uvicorn	ASGI server for FastAPI — production and development
SQLAlchemy	Full-featured ORM — models, sessions, migrations
Pydantic v2	Request/response validation and serialization with Python type hints
python-multipart	Multipart form handling for file uploads
python-dotenv	Environment variable loading from .env files
passlib (bcrypt)	Secure password hashing — bcrypt scheme
python-jose	JWT creation and verification
LangChain (langchain_community)	LLM orchestration — PromptTemplate and Ollama wrapper
Ollama (gemma3:2b)	Local LLM runtime — no cloud API dependency
PyTorch	Neural network model definition, training, and weight serialization
Pandas	CSV dataset loading for local model training
Requests	Synchronous HTTP client for OCR service calls

4.3 OCR Service

Library	Purpose
PaddleOCR	Deep-learning based OCR supporting angle correction and multi-language
OpenCV (cv2)	Image preprocessing — resizing, grayscale, denoising
Pillow	Image I/O, format conversion

NumPy	Array operations for image data
-------	---------------------------------

4.4 Federated Learning Stack

Library	Purpose
Flower (flwr)	Federated learning orchestration — server, strategy, client protocol
PyTorch	Local model training on org-specific datasets
NumPy	Parameter conversion between PyTorch tensors and Flower numpy arrays
scikit-learn	Evaluation utilities (accuracy, precision) — referenced in requirements

5. Data Flow — End-to-End

5.1 Document Upload & Processing Pipeline

The document processing pipeline is the core operational workflow of the system. It transforms a raw uploaded file into structured machine-learning records while maintaining full traceability through the blockchain log.

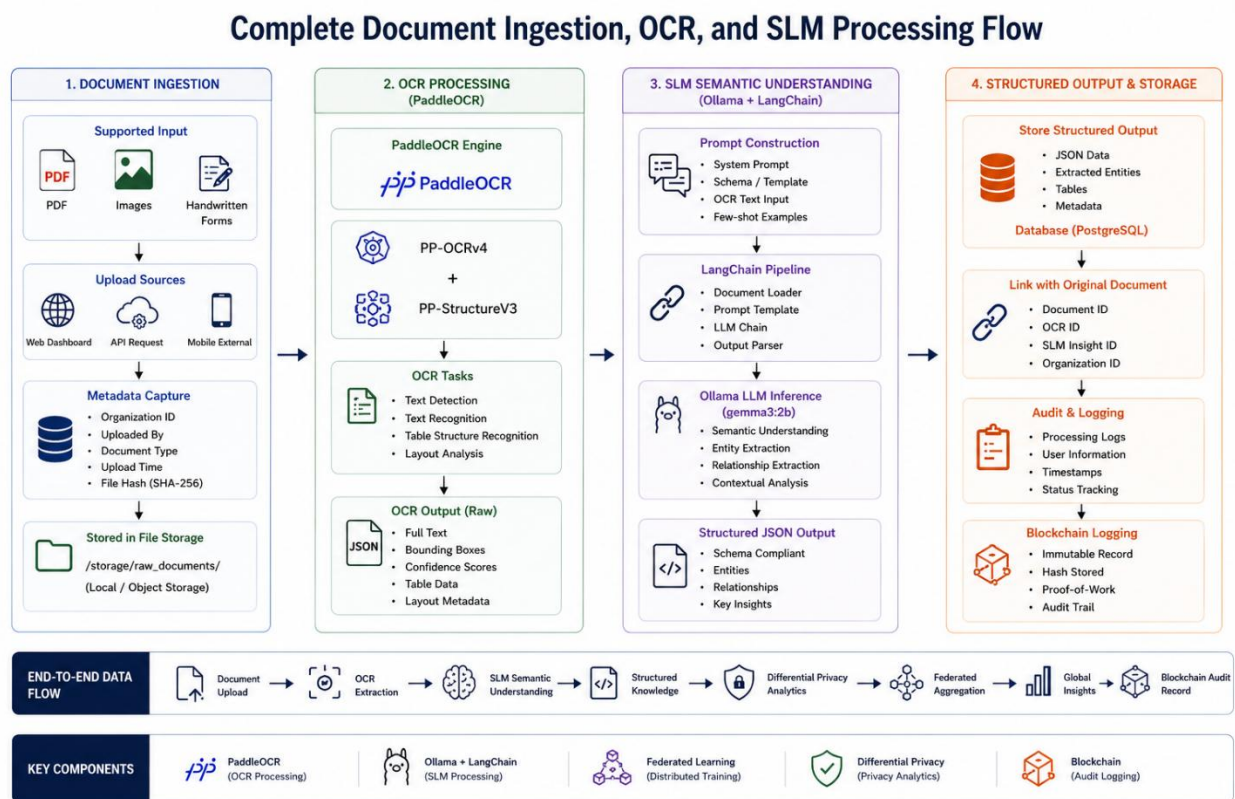


Figure 2: Complete Document Ingestion, OCR, and SLM Processing Flow

Step A — File Upload & Storage

The frontend sends a multipart POST request to /documents/upload. The backend validates the JWT token and confirms the caller holds the ORG role. The file is written to the filesystem at backend/storage/documents/org_{org_id}/{filename}. A SHA-256 hash

of the file content is computed for deduplication. If a document with the same hash already exists for that organization, processing continues on the existing record rather than creating a duplicate.

```
# Deduplication check
file_hash = hashlib.sha256(f.read()).hexdigest()
existing = db.query(Document).filter(
    Document.document_hash == file_hash,
    Document.org_id == org_id
).first()
```

Step B — OCR Text Extraction

The backend's OCR service wrapper sends an HTTP POST request to the PaddleOCR microservice running at <http://127.0.0.1:9001/ocr>. The OCR service loads the file, initializes PaddleOCR (angle classification enabled, English language mode), and returns a flat string of all detected text tokens. Performance flags disable PIR API and MKL-DNN to prevent compatibility issues in certain environments. The extracted text is stored as an OCRResult record linked to the document.

```
# OCR Engine — core extraction
result = ocr.ocr(file_path) # PaddleOCR inference
for line in result:
    for word in line:
        text = word[1][0] # (bbox, (text, confidence))
        extracted_text.append(text)
return ' '.join(extracted_text)
```

Step C — SLM Structured Extraction

This is the intelligence core of the pipeline. The extracted text passes through a two-stage extraction system:

- Stage 1 — Numeric Fast-Path: The system uses a regular expression `(\d+\.\d+|\d+)` to extract all numbers from the text. If 6 or more numbers are found, they are grouped into rows of size 6 and mapped to semantic feature names (`id`, `label`, `radius_mean`, `texture_mean`, `perimeter_mean`, `area_mean`). This path handles cancer dataset records efficiently without requiring an LLM call.
- Stage 2 — Schema-Driven LLM Extraction: If the numeric fast-path yields no results, the system fetches the schema configuration for the document type from the `SchemaConfig` table. A structured prompt is built using `LangChain PromptTemplate`, and the Ollama LLM (`gemma3:2b`) is invoked. The response is parsed to extract the first JSON array or object.

Step D — Dataset Persistence

Extracted records are deduplicated by SHA-256 hash and appended to the organization's JSON dataset file at `backend/storage/datasets/org_{org_id}.json`. This dataset accumulates across multiple document uploads and forms the training data for the federated learning clients.

Step E — Blockchain Audit Log

A blockchain entry is created for every `DOCUMENT_PROCESSED` event. The entry includes `org_id`, `action type`, `document hash`, and `filename`. The data string is cryptographically signed using the system's private key before being added as a new block to the proof-of-work chain. The block hash is also stored in the `BlockchainLog` SQL table for quick querying.

5.2 SLM Pipeline — Deep Dive

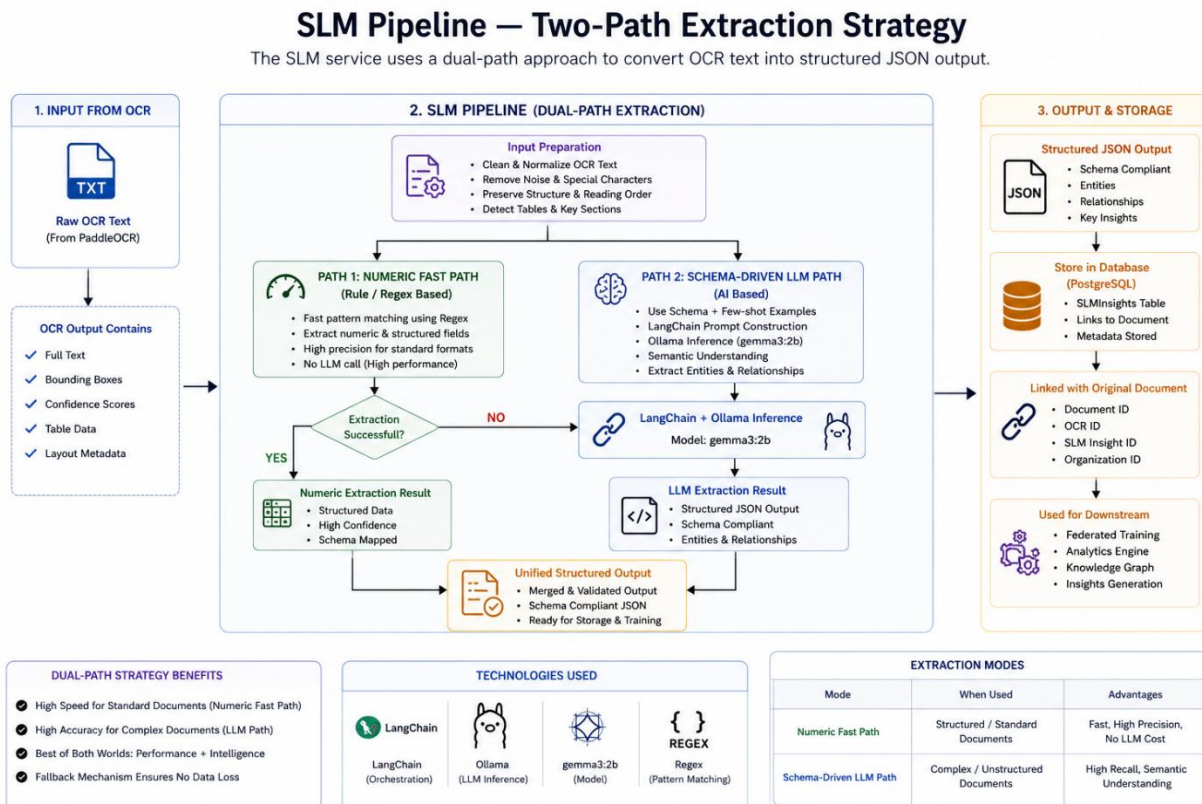


Figure 3: SLM Pipeline — Two-Path Extraction Strategy

The SLM pipeline implements a pragmatic two-path approach to structured extraction, optimizing for both performance and flexibility:

Path	When Used / Mechanism / Output
Numeric Fast-Path	Used when ≥ 6 numbers detected. Pure regex, no LLM call. Maps to fixed schema: id, label, radius_mean, texture_mean, perimeter_mean, area_mean. Ideal for structured CSV-formatted documents.
Schema-Driven LLM	Used for non-numeric or ambiguous text. Fetches doc_type schema from DB. Builds a LangChain PromptTemplate instructing gemma3:2b to return valid JSON. Extracts first JSON array/object from response.

Fallback

If schema is not found in DB, returns [{raw_text: cleaned_text[:300]}] — preserving data without blocking the pipeline.

6. Federated Learning Architecture

The federated learning subsystem enables multiple organizations to collaboratively improve a shared predictive model without centralizing their sensitive datasets. The implementation uses the Flower (flwr) framework with a FedAvg aggregation strategy.

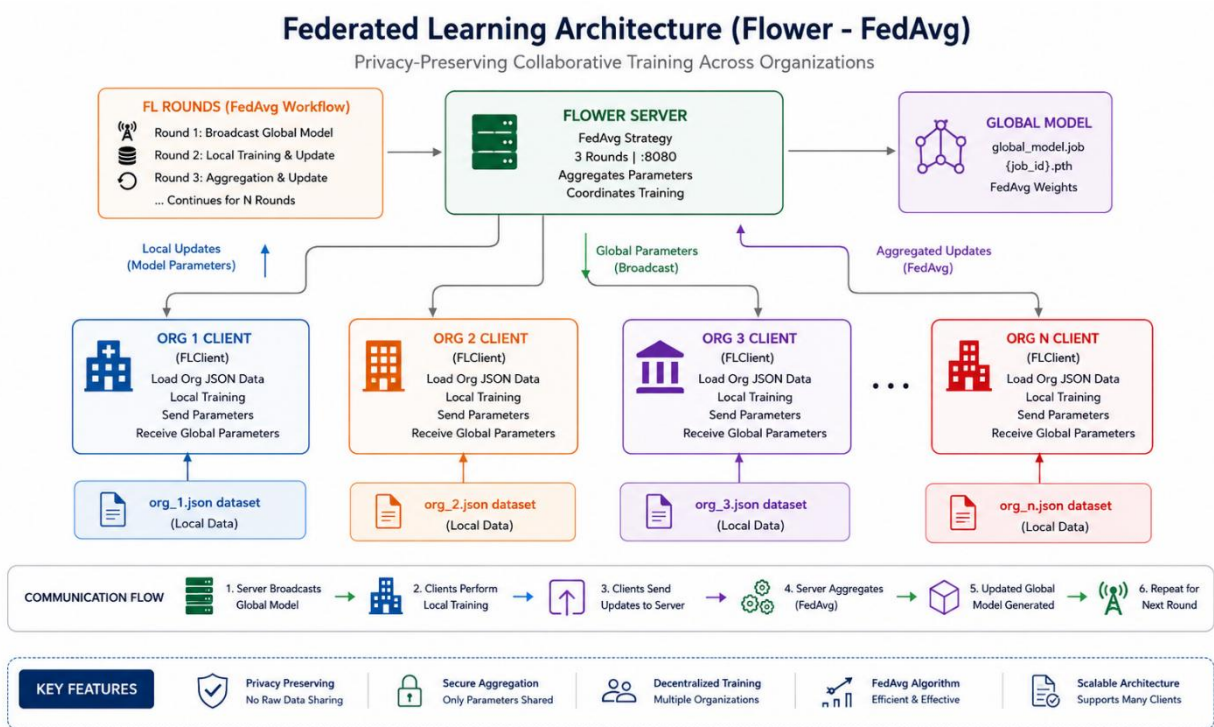


Figure 4: Federated Learning Architecture — Flower FedAvg with N Organization Clients

6.1 Flower Server

The Flower server runs at 127.0.0.1:8080 and orchestrates training across all available clients. It uses the standard FedAvg strategy with the following configuration:

Parameter	Value / Description
-----------	---------------------

<code>fraction_fit</code>	1.0 — All available clients participate in every round
<code>min_fit_clients</code>	1 — At minimum one client is required to proceed
<code>min_available_clients</code>	1 — Training begins once at least one client connects
<code>num_rounds</code>	3 — Three rounds of federated aggregation per training session
<code>server_address</code>	127.0.0.1:8080
<code>aggregatio</code>	FedAvg — Weighted average of client parameter updates

6.2 Flower Client (FLClient)

Each organization runs its own FLClient, a subclass of `fl.client.NumPyClient`. The client is initialized with the organization's ID and immediately loads the org's JSON dataset from `backend/storage/datasets/org_{org_id}.json`. Feature count is determined dynamically from the dataset, allowing flexible model sizing.

```
class FLClient(fl.client.NumPyClient):
    def __init__(self, org_id):
        self.X, self.y = load_data(org_id)
        input_size = self.X.shape[1]
        self.model = get_model(input_size)  # SimpleModel(input_size)

    def fit(self, parameters, config):
        self.set_parameters(parameters)      # Receive global weights
        train_local_model(self.model, self.X, self.y)  # Local training
        return self.get_parameters(config), len(self.X), {}
```

6.3 Model Architecture

Both the local training path and the federated learning path use the same simple linear model architecture. This design choice prioritizes interpretability and efficiency for structured tabular data:

```

class SimpleModel(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.fc = nn.Linear(input_size, 2)  # Binary classification

    def forward(self, x):
        return self.fc(x)  # 2 output logits

```

The model outputs two logits for binary classification (e.g., benign vs. malignant in the cancer use case). Training uses CrossEntropyLoss and the Adam optimizer at a learning rate of 0.001 for 50 epochs in the local training path. The federated client uses the same optimizer for its local training rounds.

6.4 Weight Aggregation (FedAvg + Custom)

The standard FedAvg strategy implemented by Flower averages model parameters weighted by the number of samples each client contributed. The repository also contains a custom aggregation utility (`federated_service.py::average_two_models`) that performs a simple two-model averaging for the backend-integrated update path:

```

def average_two_models(global_weights, local_weights):
    for key in global_weights.keys():
        if global_weights[key].shape != local_weights[key].shape:
            raise Exception(f'Shape mismatch for {key}')
        new_weights[key] = (global_weights[key] + local_weights[key]) / 2
    return new_weights

```

6.5 Dataset Loading for Federated Clients

The federated dataset loader (`federated/client/dataset.py`) reads the JSON dataset for the given org, dynamically detects all numeric feature columns (excluding id and label), and constructs PyTorch tensors for training:

- Features: All numeric columns except id and label — detected dynamically, so no hardcoded schema is needed.

- Target: The label column, cast to torch.long for cross-entropy classification.
- Data augmentation: Not currently implemented — raw extracted records are used directly.

7. Blockchain Audit Logging

The system implements a custom proof-of-work blockchain for immutable audit logging. Every significant system action — document processing, model training initiation, federated weight updates — is recorded as a cryptographically linked block.

7.1 Block Structure

Field	Description
index	Sequential block number in the chain
timestamp	UTC ISO timestamp of block creation
data	JSON-serialized action payload (org_id, action, doc_hash, details, signature)
previous_hash	SHA-256 hash of the preceding block (chain linkage)
nonce	Proof-of-work nonce — incremented until hash meets difficulty target
hash	SHA-256 of index + timestamp + data + previous_hash + nonce + signature
signature	ECDSA/RSA digital signature of the data string using system private key

7.2 Proof of Work

Each block requires a proof-of-work computation (backend/blockchain/proof.py) where the nonce is incremented until the block hash satisfies the configured difficulty requirement. This prevents retroactive tampering with block data.

7.3 Chain Validation

After every block addition, the full chain is validated. Two checks are applied:

- Hash integrity: Each block's stored hash is recalculated and compared — detects data tampering within a block.
- Chain linkage: Each block's `previous_hash` must equal the actual hash of the preceding block — detects insertion or reordering of blocks.

If validation fails after adding a block, the block is rolled back and an exception is raised, protecting the chain's integrity.

```
def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        prev = self.chain[i - 1]
        if current.hash != current.calculate_hash(): return False # Hash
tampered
        if current.previous_hash != prev.hash: return False #
Chain broken
    return True
```

7.4 Logged Action Types

Action Type	Trigger Event
DOCUMENT_PROCESSED	Successful document upload, OCR, and SLM extraction
GLOBAL_MODEL_INITIALIZED	First federated update — global model created from org's local model
GLOBAL_MODEL_UPDATED	Subsequent federated updates — FedAvg applied to global model
AUTH events (planned)	Login, registration, role changes

8. API Reference

The FastAPI backend exposes a comprehensive REST API organized into feature-specific routers. All endpoints (except `/auth/*`) require a valid JWT bearer token. Role-based access is enforced via the `require_role()` dependency.

8.1 Authentication (`/auth`)

Method + Path	Request Body	Description
POST <code>/auth/register</code>	{email, password, role, org_id?}	Register new user. Role must be ORG or RESEARCHER. Returns pending-approval message.
POST <code>/auth/login</code>	{email, password}	Authenticate and receive JWT token. Returns {access_token}. Fails if not approved.

8.2 Document Management (`/documents`)

Method + Path	Auth Role	Description
POST <code>/documents/upload</code>	ORG	Upload a document file (multipart). Triggers OCR → SLM → dataset save → blockchain log pipeline.

8.3 SLM Endpoint (`/slm`)

Method + Path	Request Body	Description
POST /slm/analyze	{text, doc_type?}	Directly invoke the SLM extraction pipeline on provided text. doc_type defaults to 'cancer'.

8.4 Schema Management (/schema)

Method + Path	Auth Role	Description
POST /schema/create	ADMIN	Create a new SchemaConfig with core_fields and dynamic_fields JSON.
GET /schema/{doc_type}	Any	Retrieve schema definition for a specific document type.
PUT /schema/{doc_type}	ADMIN	Update an existing schema definition.

8.5 Training Endpoint (/train)

Method + Path	Auth Role	Description
POST /train/local	ADMIN, RESEARCHER, ORG,	Initiate local model training. Parameters: dataset_path, doc_type, org_id, job_id. Returns model save path.

8.6 Blockchain (/blockchain)

Method + Path	Auth Role	Description
---------------	-----------	-------------

GET /blockchain/logs	ADMIN, RESEARCHER	Retrieve all blockchain log entries from the SQL table.
GET /blockchain/chain	ADMIN	Return the full raw blockchain as JSON array of blocks.

8.7 Admin (/admin)

Method + Path	Auth Role	Description
GET /admin/users	ADMIN	List all registered users.
PUT /admin/approve/{user_id}	ADMIN	Approve a pending user registration.
PUT /admin/grant-download/{user_id}	ADMIN	Grant a user permission to download the global model.

8.8 Organization (/org)

Method + Path	Auth Role	Description
GET /org/dashboard	ORG	Return summary stats for the authenticated org: document count, dataset size, model jobs.
GET /org/dataset	ORG	Download the org's current dataset as JSON.

9. Database Schema

The system uses SQLAlchemy with SQLite (development) or PostgreSQL (production). The schema is auto-generated via `init_db()` on startup using declarative Base models.

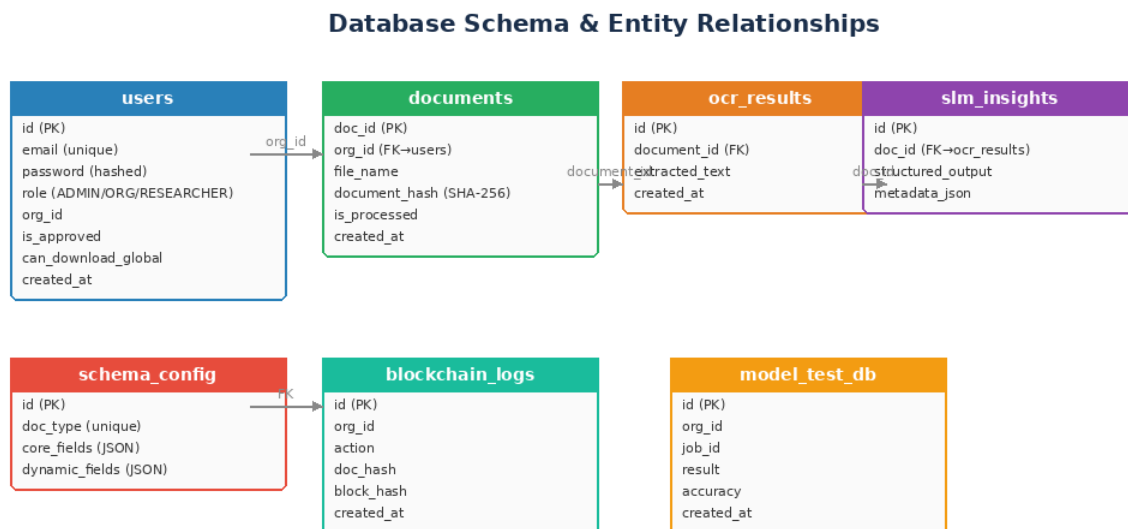


Figure 5: Database Entity-Relationship Diagram

9.1 Users Table

Column	Type / Constraints / Notes
id	Integer, PK, auto-increment
email	String, UNIQUE, NOT NULL — login identifier
password	String, NOT NULL — bcrypt hashed, never stored plain
role	String — ADMIN ORG RESEARCHER

org_id	Integer, nullable — only set for ORG users; auto-incremented on registration
is_approved	Boolean, default False — must be approved by ADMIN before login works
can_download_global	Boolean, default False — ADMIN can grant global model download access
created_at	DateTime, default utcnow

9.2 Documents Table

Column	Type / Constraints / Notes
doc_id	Integer, PK
org_id	Integer, FK → users.org_id
file_name	String — original uploaded filename
document_hash	String — SHA-256 hex digest for deduplication
is_processed	Boolean — set True after full pipeline completion
created_at	DateTime

9.3 OCR Results Table

Column	Type / Notes
id	Integer, PK
document_id	Integer, FK → documents.doc_id
extracted_text	Text — full raw OCR output
created_at	DateTime

9.4 SLM Insights Table

Column	Type / Notes
id	Integer, PK
doc_id	Integer, FK → ocr_results.id
structured_output	Text — stringified extracted records or JSON array
metadata_json	String — reserved for future metadata (currently '{}')

9.5 Schema Config Table

Column	Type / Notes
id	Integer, PK
doc_type	String, UNIQUE — e.g., 'cancer'
core_fields	Text — JSON object: {field_name: dtype}. dtype='target' marks the label field.
dynamic_fields	Text — JSON for additional configurable fields

9.6 Blockchain Logs Table

Column	Type / Notes
id	Integer, PK
org_id	Integer — which organization triggered the action
action	String — e.g., DOCUMENT_PROCESSED, GLOBAL_MODEL_UPDATED
doc_hash	String, nullable — the document's SHA-256 hash
block_hash	String — SHA-256 hash of the corresponding blockchain block

created_at	DateTime
------------	----------

10. Security Architecture

10.1 Authentication Flow

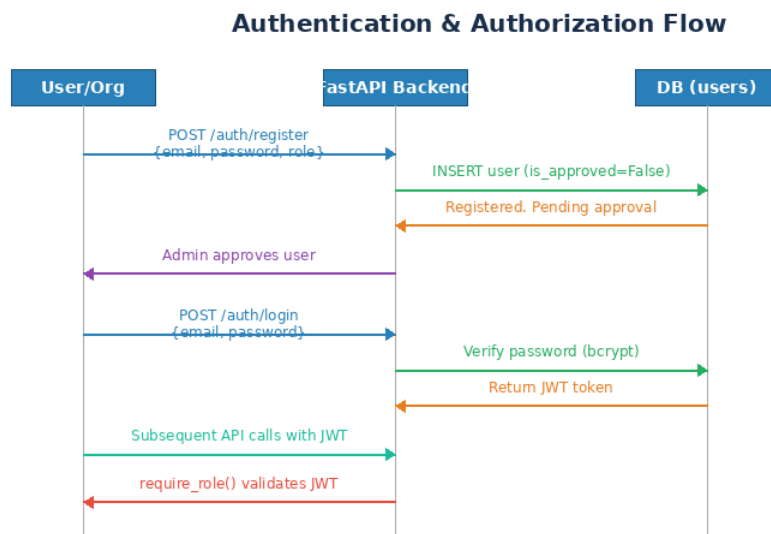


Figure 6: Authentication & Authorization Flow (Sequence Diagram)

Authentication is implemented using JWT (JSON Web Tokens) with HS256 signing. The workflow is:

5. User registers with email, password, and role. The password is hashed with bcrypt before storage.
6. An ADMIN must approve the user by setting `is_approved=True` before they can log in.
7. On login, the backend verifies the bcrypt password hash and checks approval status.
8. A signed JWT is issued containing `user_id`, `role`, `org_id`, and `can_download_global` claims, valid for 10 hours.
9. All subsequent API calls include the token in the Authorization: Bearer header.

10. The `require_role()` FastAPI dependency decodes the token and enforces role restrictions per endpoint.

10.2 Role-Based Access Control

Endpoint Group	ADMIN	ORG	RESEARCHER
/auth/*	All users	All users	All users
/documents/upload	—	YES	—
/train/local	YES	YES	YES
/schema/*	YES (write)	Read only	Read only
/admin/*	YES	—	—
/blockchain/*	YES	—	YES (logs)
/org/*	—	YES	—
/model-test/*	—	—	YES

10.3 Data Security Considerations

- File deduplication via SHA-256 prevents duplicate processing and storage waste.
- Each organization's data is stored in isolated directories (`org_{org_id}/`) preventing cross-org data access.
- The blockchain digital signature (`crypto.py` + `keys.py`) ensures action logs cannot be silently forged.
- JWT secret key (currently 'secret') should be replaced with a cryptographically secure random value in production.
- Database passwords should be managed via environment variables (`.env` file) — not committed to source control.

Security Recommendations for Production

1. Replace `SECRET_KEY` in `auth_service.py` with a 256-bit random value from `os.urandom(32).hex()`
2. Configure CORS origins explicitly — do not use wildcard (*) in production
3. Enable HTTPS via a reverse proxy (Nginx/Caddy) — all API and file transfers should be TLS-encrypted
4. Consider adding rate limiting to `/auth/login` to prevent brute-force attacks
5. Rotate JWT signing keys periodically and implement token revocation for logout flows

11. File Storage Layout

The filesystem storage layer is organized to enforce per-organization isolation and to clearly separate documents, datasets, and model weights:

```
backend/storage/
├── documents/
│   ├── org_1/                # Uploaded documents for Org 1
│   │   ├── patient_report.pdf
│   │   └── scan_data.jpg
│   └── org_2/                # Uploaded documents for Org 2
│       └── clinical_record.png
├── datasets/
│   ├── org_1.json            # Accumulated JSON records for Org 1
│   ├── org_1.csv             # CSV version for local model training
│   └── org_2.json
├── local_models/
│   ├── org_1/
│   │   └── local_model_job_42.pth
│   └── org_2/
│       └── local_model_job_43.pth
└── global_model_job_42.pth    # Aggregated global model
```

Path Pattern	Contents / Format
storage/documents/org_{id}/{file}	Original uploaded files (PDF, PNG, JPG, etc.)
storage/datasets/org_{id}.json	JSON array of extracted records — growing dataset

storage/datasets/org_{id}.csv	CSV version — used by local model training (pd.read_csv)
storage/local_models/org_{id}/local_model_job_{job}.pth	PyTorch state dict — org-specific locally trained model
storage/global_model_job_{job}.pth	Aggregated global model weights (FedAvg result)

12. Frontend Architecture

The React SPA is organized around role-specific page modules, shared components, and a context/state layer. Routing is protected — users are redirected to the login page if they attempt to access routes for which they lack the required role.

12.1 Route Structure

Route	Component / Role Access
/admin/dashboard	AdminDashboard — ADMIN only
/admin/users	UserManagement — approve/reject registrations, grant permissions
/admin/schema	SchemaBuilder — define document type schemas
/admin/jobs	JobManagement — monitor training jobs
/admin/downloads	AdminDownloadsPage — model download management
/org/dashboard	OrgDashboard — org overview and stats
/org/upload	DocumentUpload — drag-and-drop document submission
/org/training	TrainingPage — initiate local/federated training
/org/downloads	DownloadsPage — download global model if authorized
/org/jobs	JobBoard — view job status
/researcher/*	Analytics, model testing, dataset inspection pages

12.2 State Management

Zustand is used for global client state. The auth store holds the decoded JWT payload (user_id, role, org_id, can_download_global) and manages login/logout state. API calls use Axios with an interceptor that injects the Authorization: Bearer {token} header on every request, eliminating repetitive token handling in individual components.

12.3 Key UI Features

- Document Upload: A react-dropzone area supporting drag-and-drop. Files are submitted as FormData. Real-time progress and result toasts are shown via react-hot-toast.
- Schema Builder: An interactive ADMIN form for defining document type schemas. Allows adding core fields with data type annotations (float, int, target). Submitted to /schema/create.
- Analytics Dashboard: Recharts-based bar and line charts visualizing document processing volumes, extraction success rates, and model accuracy over training jobs.
- Blockchain Viewer: A table of all blockchain log entries showing org_id, action, timestamp, and block hash for full audit trail inspection.

13. OCR Microservice

The OCR service is a standalone FastAPI application (`ocr_service/app.py`) running on port 9001. It is architecturally separate from the main backend to allow independent scaling, replacement, or upgrade without affecting the rest of the system.

13.1 PaddleOCR Configuration

Parameter	Value / Notes
<code>use_angle_cls</code>	True — enables detection of rotated/skewed text (angle classifier)
<code>lang</code>	'en' — English language model
<code>show_log</code>	False — suppresses verbose PaddleOCR logs
<code>FLAGS_enable_pir_api</code>	0 — disabled to prevent Paddle 2.6.x compatibility errors
<code>FLAGS_use_mkl_dnn</code>	0 — MKL-DNN disabled for stability in Docker/sandbox environments

13.2 Text Extraction Process

The OCR engine processes files through the following steps:

11. File existence validation — raises `FileNotFoundError` for missing files.
12. PaddleOCR inference — returns a nested list of detected text regions:
[[`[bbox_points]`, (`text`, `confidence`)], ...].
13. Text token collection — iterates over all regions and accumulates detected text strings.
14. Join and return — all tokens are joined with spaces to form a single text string.

The OCR service is initialized once at module load time (not per-request) to avoid expensive model loading on every call. This significantly improves throughput for high-volume document processing workloads.

13.3 Service Interface

```
# POST http://127.0.0.1:9001/ocr
# Content-Type: multipart/form-data
# Field: file (binary)
#
# Response: { "text": "extracted text string" }
```

14. Deployment & Configuration

14.1 Environment Variables

Variable	Default / Notes
OLLAMA_MODEL	gemma3:2b — Ollama model identifier
OLLAMA_BASE_URL	http://localhost:11434 — Ollama server URL
DATABASE_URL	postgresql://user:password@localhost/dbname — SQLAlchemy connection string
STORAGE_PATH	backend/storage/documents — root path for uploaded files
SECRET_KEY	'secret' (⚠ must be changed in production)

14.2 Service Startup Sequence

The following sequence is required to bring up the full system:

15. Start Ollama and pull the model: `ollama serve && ollama pull gemma3:2b`
16. Start the OCR service: `uvicorn ocr_service.app:app --port 9001`
17. Start the FastAPI backend: `uvicorn backend.main:app --reload --port 8000`
18. Start the frontend dev server: `cd frontend && npm run dev (port 5173)`
19. (Optional) Start the Flower FL server: `python -m federated.server.server`
20. (Optional) Start FL clients per org: `python -m federated.client.client {org_id}`

14.3 Docker Compose

A `docker-compose.yml` is provided in the `docker/` directory for containerized deployment. The compose file orchestrates the following services: the FastAPI backend, the OCR service, the PostgreSQL database, and the frontend Nginx server. Ollama should be run

with GPU access on the host and exposed to the backend container via the OLLAMA_BASE_URL environment variable.

14.4 Python Requirements

requirements_backend.txt	requirements_fl.txt	requirements_ocr.txt
fastapi	flwr (Flower)	paddleocr
uvicorn	torch	opencv-python
sqlalchemy	numpy	pillow
pydantic	scikit-learn	numpy
python-multipart		
python-dotenv		
langchain_community		
passlib / python-jose		
requests / pandas		

15. Training Pipeline — Local & Federated

15.1 Local Training Workflow

Local model training is triggered via the POST `/train/local` endpoint. The workflow:

21. Schema features are fetched from SchemaConfig: `core_fields` where `dtype != 'target'` become feature columns; the field with `dtype == 'target'` is the label column.
22. The dataset CSV is loaded using `pd.read_csv(dataset_path)`. Column presence is validated against the schema.
23. Feature and target arrays are converted to PyTorch tensors (float32 for X, long for y).
24. A `SimpleModel(input_size)` is instantiated with `CrossEntropyLoss` and Adam optimizer (`lr=0.001`).
25. Training runs for 50 epochs in a standard forward → loss → backward → step loop.
26. The model state dict is saved to `backend/storage/local_models/org_{org_id}/local_model_job_{job_id}.pth`.

15.2 Format Compatibility

An important architectural note: the local training path requires CSV input (via `pd.read_csv`), while the federated learning clients consume JSON (`org_{org_id}.json`). The `dataset_service` provides a `save_csv_dataset()` function that converts the JSON dataset to CSV format before local training is initiated. This format bridge is critical to ensure both paths can consume the same underlying data.

Training Path	Dataset Format / Location
Local (<code>/train/local</code>)	CSV — <code>backend/storage/datasets/org_{id}.csv</code>

Federated (FL Client)	JSON — backend/storage/datasets/org_{id}.json
Data Source	Both formats derived from the same extracted records

15.3 Federated vs. Local Training Comparison

Dimension	Local Training vs. Federated Learning
Data Privacy	Local: data leaves the org storage. Federated: only model weights shared
Coordination	Local: single org, no server needed. Federated: requires Flower server at :8080
Model Updates	Local: trains from scratch each job. Federated: updates from global model
Aggregation	Local: no aggregation. Federated: FedAvg across all participating orgs
Scalability	Local: O(1) orgs. Federated: scales to N orgs simultaneously
Use Case	Local: testing, debugging. Federated: production privacy-preserving training

16. Known Limitations & Future Work

16.1 Current Limitations

- The SLM numeric fast-path is hardcoded to a 6-column schema (cancer dataset). Documents with different structures will fall through to the LLM path, which has higher latency.
- The SimpleModel architecture (single linear layer) is appropriate for linear problems but will underperform on non-linear feature spaces. More complex architectures (MLPs, attention heads) would improve accuracy.
- The Flower federated server and backend API are not integrated — triggering federated training requires manual CLI invocation of the server and client processes.
- The JWT secret key is hardcoded as 'secret' in the source code — a significant security risk for any non-development deployment.
- The blockchain does not persist chain state across server restarts in a production-safe way — the full chain should be loaded from persistent storage with integrity verification on startup.
- No differential privacy mechanisms (e.g., Gaussian noise injection) are applied to federated model updates, which could allow gradient leakage attacks.

16.2 Planned Improvements

- Pydantic validation of LLM output — enforce schema compliance on extracted JSON before storing.
- Table detection in OCR post-processing — use bounding box coordinates from PaddleOCR to reconstruct tabular structures more accurately.
- Federated client orchestration via backend — trigger FL rounds via the /train API rather than manual CLI startup.

- Differential privacy integration — add noise injection to federated gradients using Opacus or similar library.
- Model evaluation endpoint expansion — implement F1, AUC-ROC, and confusion matrix reporting in /model-test.
- Production blockchain persistence — implement chain serialization/deserialization with integrity verification on server startup.
- Support for additional document types — extend schema builder to support arbitrary field types, not just the cancer dataset template.
- GPU acceleration — containerize with CUDA support for faster OCR inference and model training at scale.

17. Conclusion

The Federal-SLM-System represents a comprehensive proof-of-concept for privacy-preserving federated document intelligence. By combining state-of-the-art deep learning OCR (PaddleOCR), a pragmatic dual-path structured extraction pipeline (SLM with Ollama/LangChain), and a robust federated learning framework (Flower FedAvg), the system enables organizations to extract structured insights from documents and collaborate on AI model training — while retaining full data sovereignty.

The addition of a cryptographically signed, proof-of-work blockchain audit log addresses the auditability requirements of regulated industries, ensuring that every processing action is traceable, tamper-evident, and independently verifiable.

The system's modular microservice architecture — with clear separation between the frontend, backend API, OCR service, and FL components — provides a solid foundation for scaling each subsystem independently and for swapping components as better alternatives emerge (e.g., replacing gemma3:2b with a fine-tuned domain-specific model, or upgrading the SimpleModel to a deep MLP).

With the security hardening, model architecture improvements, and FL-backend integration outlined in the future work section, this system has strong potential for production deployment in healthcare, legal, and financial document processing workflows.

Repository: <https://github.com/thanushshetty9353/Federal-SLM-System>